Chat with Energy Documents (RAG)

Course Name:Gen AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	**MAHAK SHRIMAL**	**EN22CS301566**
02	**MAHAK TAOSE**	**EN22CS301567**
03	**NAINA MANGHANI**	**EN22CS301626**
04	**MOHIT PATEL**	**EN22CS301610**
05	**PAAVAN PATNI**	**EN22CS301671**

Group Name:02D6

Project Number:GAI-13

Industry Mentor Name:

University Mentor Name: Prof. Ajaj Khan

Academic Year: 2025-2026

TABLE OF CONTENTS

S.NO.	TOPICS	
1	INTRODUCTION	1.1 SCOPE OF THE DOCUMENT
		1.2 INTENDED AUDIENCE
		1.3 SYSTEM OVERVIEW
2	SYSTEM DESIGN	2.1 APPLICATION DESIGN
		2.2 PROCESS FLOW
		2.3 INFORMATION FLOW
		2.4 COMPONENTS DESIGN
		2.5 KEY DESIGN CONSIDERATIONS
		2.6 API CATALOGUE
3.	DATA DESIGN	3.1 DATA MODEL
		3.2 DATA ACCESS MECHANISM
		3.3 DATA RETENTION POLICIES
		3.4 DATA MIGRATION
4.	INTERFACES	4.1 USER INTERFACE
		4.2 BACKEND INTERFACE
		4.3 EXTERNAL APIs
5.	STATE AND SESSION MANAGEMENT	
6.	CACHING	
7.	FUNCTIONAL REQUIREMENTS	7.1 DOCUMENT UPLOAD
		7.2 DOCUMENT PROCESSING
		7.3 EMBEDDING GENERATION

		7.4 QUERY PROCESSING
8.	**NON-FUNCTIONAL REQUIREMENTS**	8.1 SECURITY
		8.2 PERFORMANCE
		8.3 SCALABILITY
		8.4 RELIABILITY
9.	**REFERENCES**	

1. INTRODUCTION

The rapid growth of domain-specific data, especially in the energy sector, has created a need for intelligent systems that can efficiently extract insights from large volumes of documents. Traditional keyword-based search systems fail to provide context-aware answers, making it difficult for users to retrieve precise information from technical reports and manuals. To address this limitation, this project implements a Retrieval-Augmented Generation (RAG) based system that enables users to interact with energy-related documents through natural language queries.The system integrates modern Generative AI technologies, including Large Language Models (LLMs), vector databases, and embedding techniques, to provide accurate and context-aware responses. Users can upload PDF documents, which are processed into structured chunks and stored as vector embeddings. During querying, the system retrieves relevant chunks and generates responses strictly based on document content.The architecture is designed using Python-based backend technologies such as FastAPI, along with vector databases like ChromaDB and APIs such as OpenAI for embeddings and answer generation. This High-Level Design document outlines the architecture, workflow, components, and system interactions.

1.1 SCOPE OF THE DOCUMENT

This document defines the High-Level Design of the "Chat with Energy Documents" system based on the RAG architecture. It focuses on describing system structure, workflows, and interactions between components without going into detailed implementation.

1. Defines overall system architecture for RAG pipeline
2. Explains integration of document processing, vector storage, and LLM
3. Describes data flow from document upload to answer generation
4. Covers system modules such as backend APIs, vector DB, and LLM
5. Highlights deployment-ready architecture using Python and APIs

1.2 INTENDED AUDIENCE

This document is intended for stakeholders involved in development, evaluation, and usage of the system.

1. **Developers –** To understand system architecture and integration
2. **AI/ML Engineers –** To analyze embedding and retrieval mechanisms
3. **Project Evaluators –** To assess design completeness and feasibility
4. **Students –** For academic understanding and viva preparation

1.3 SYSTEM OVERVIEW

The system is an AI-powered document assistant that enables users to upload energy-related PDFs and ask questions in natural language. It uses a Retrieval-Augmented Generation pipeline to ensure answers are grounded in uploaded documents.

1. User uploads PDF document
2. System extracts text and splits into chunks
3. Chunks are converted into vector embeddings

4. Embeddings are stored in vector database (ChromaDB)
5. User asks a query
6. System retrieves relevant chunks
7. LLM generates context-aware answer

2. SYSTEM DESIGN

System design is the process of planning and structuring how a system will work, including its components, architecture, data flow, and interactions.

2.1 APPLICATION DESIGN

The application follows a backend-driven architecture using FastAPI, with integration of document processing utilities, vector database, and LLM APIs.

1. **Backend Layer (FastAPI)-**Handles document upload and query APIs.Manages request-response lifecycle
2. **Processing Layer-**Extracts text using PDF processing libraries.Splits text into chunks
3. **Embedding Layer-**Converts text chunks into vector embeddings using OpenAI
4. **Storage Layer-**Stores embeddings in ChromaDB vector database
5. **Retrieval + LLM Layer-**Retrieves relevant chunks.Generates final answer using LLM

2.2 PROCESS FLOW

The system follows a structured pipeline from document ingestion to answer generation.

1. User uploads PDF through API
2. File is processed and text is extracted
3. Text is divided into chunks
4. Each chunk is converted into embeddings
5. Embeddings are stored in vector DB
6. User submits query
7. Query is converted into embedding
8. Similar chunks are retrieved
9. LLM generates response

2.3 INFORMATION FLOW

The system processes information in a sequential and structured manner.

1. **Input** → PDF document
2. **Processing** → Text extraction & chunking
3. **Embedding** → Vector representation
4. **Storage** → Vector DB (ChromaDB)
5. **Query** → Retrieval mechanism
6. **Output** → LLM-generated answer

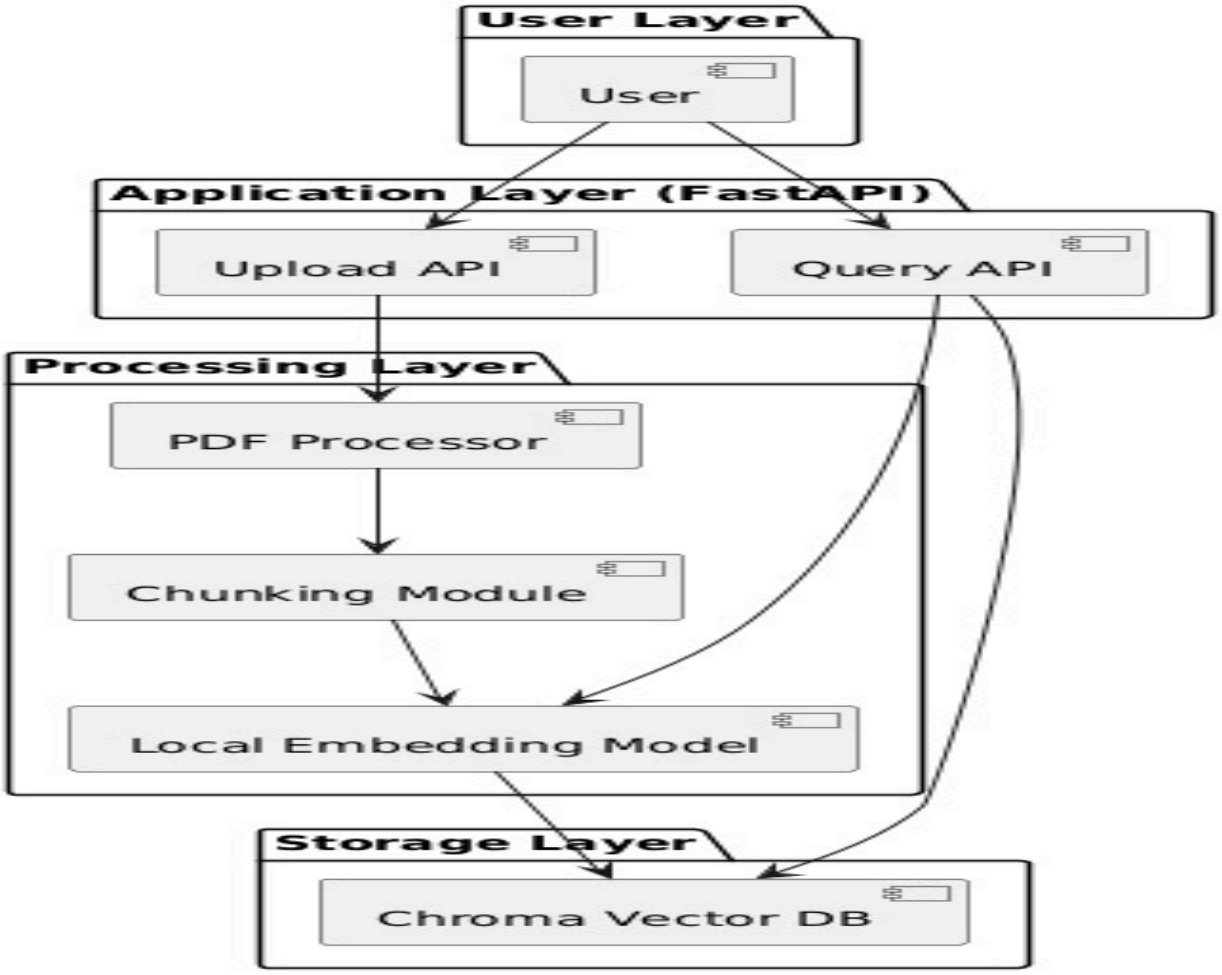

Figure 1:High-Level System Architecture

2.4 COMPONENTS DESIGN

The system consists of multiple interacting components:

1. FastAPI Backend-Handles APIs (/upload, /query)
2. Document Processor-Extracts and chunks text
3. Embedding Generator-Converts text to vectors
4. Vector Store (ChromaDB)-Stores embeddings
5. Retrieval Module-Fetches relevant chunks
6. LLM API-Generates final answer

2.5 KEY DESIGN CONSIDERATIONS

1. **Accuracy** – Answers must be grounded in documents
2. **Scalability** – Handle multiple documents/projects
3. **Efficiency** – Optimize chunking and retrieval
4. **Modularity** – Independent components

5. **Security –** Protect API keys and data

2.6 API CATALOGUE

1. **OpenAI Embedding API –** Generate embeddings
2. **OpenAI Chat API –** Generate answers
3. FastAPI Endpoints:

A. **/upload** → Upload documents
B. **/query** → Ask questions
C. **/health** → Check system status

3. DATA DESIGN

The system uses a document-centric data model focused on embeddings and retrieval.

3.1 DATA MODEL

1. **Document –** Raw PDF text
2. **Chunk –** Smaller text segments
3. **Embedding –** Vector representation
4. **Query –** User input
5. **Response –** Generated answer

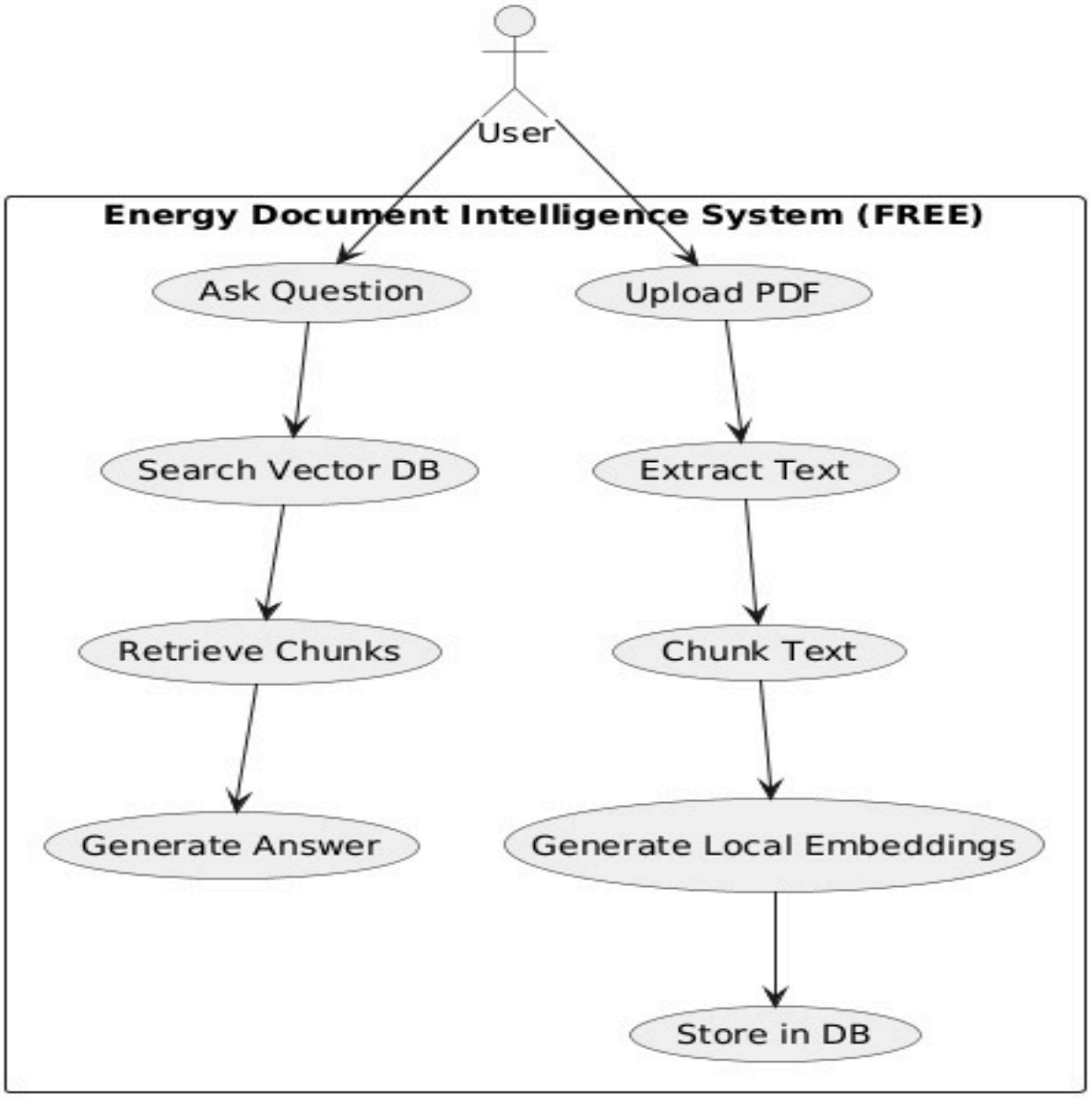

Figure 2 :Use Case Diagram

1. **User Actions**

The user can do two main things:

A. Upload PDF
B. Ask Question

2. **When User Uploads PDF**

A. PDF is uploaded
B. Text is extracted from the document
C. Text is split into smaller chunks
D. Each chunk is converted into embeddings (vector format)
E. Embeddings are stored in the vector database

3. **When User Asks Question**

A. User enters a query
B. System searches the vector database
C. Retrieves the most relevant chunks
D. Generates an answer using those chunks

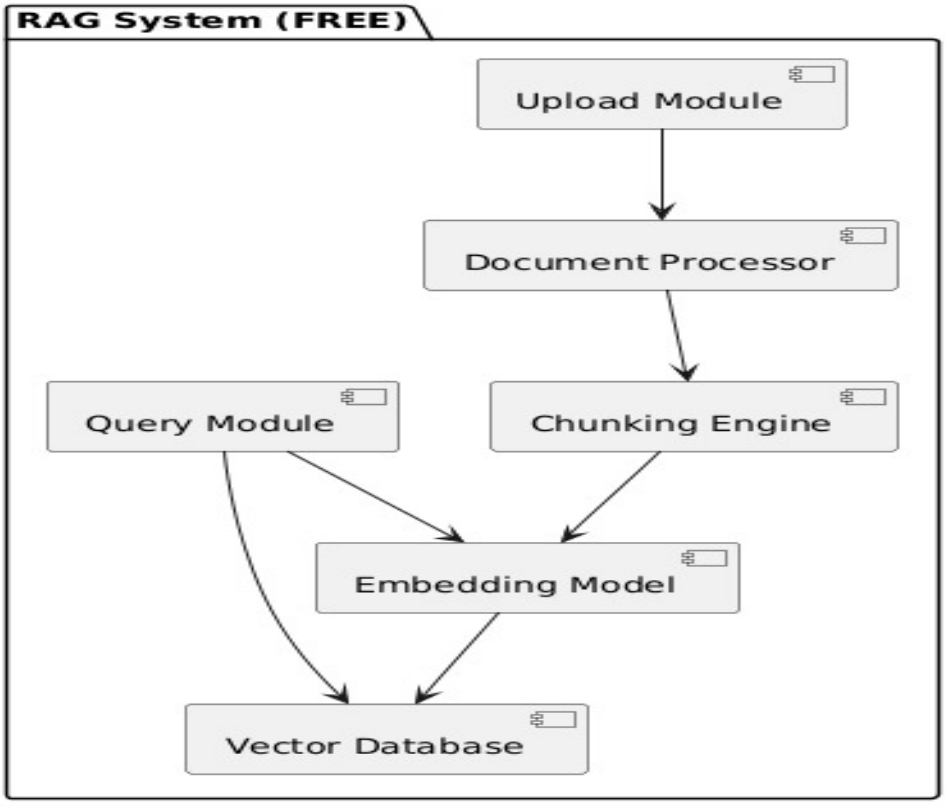

Figure 3 :Component Diagram

Flow of the System

1. **Upload Module-**User uploads the PDF document
2. **Document Processor-**Extracts text from the uploaded file
3. **Chunking Engine-**Splits the text into smaller chunks
4. **Embedding Model-**Converts chunks (and queries) into vector format
5. **Vector Database-**Stores embeddings and retrieves relevant data
6. **Query Module-**Takes user question and sends it for processing

Overall Working

1. Upload flow: PDF → text → chunks → embeddings → stored in DB
2. Query flow: Question → embedding → search DB → retrieve answer

3.2 DATA ACCESS MECHANISM

1. Vector DB uses similarity search
2. Retrieval based on cosine similarity
3. Top-k chunks are selected

3.3 DATA RETENTION POLICIES

1. Embeddings stored locally
2. Temporary files deleted after processing
3. Queries are not permanently stored

3.4 DATA MIGRATION

1. Can migrate from ChromaDB to Pinecone
2. Embeddings can be exported/imported
3. Supports scaling to cloud DB

4. INTERFACES

Interfaces define the points of interaction between different components of a system or between the user and the system.

4.1 USER INTERFACE

1. Web UI (Streamlit or frontend)
2. Allows file upload and query input

4.2 BACKEND INTERFACE

1. REST APIs using FastAPI
2. Handles communication between modules

4.3 EXTERNAL APIs

1. OpenAI APIs
2. Vector DB APIs

5. STATE AND SESSION MANAGEMENT

1. Vector DB maintains document state
2. No persistent user sessions
3. Project-based isolation supported

6. CACHING

1. Embeddings stored to avoid recomputation
2. Retrieval results optimized
3. Improves response time

7. FUNCTIONAL REQUIREMENTS

Functional requirements describe what a system should do,the specific features, operations, and behaviors that the system must perform to meet user needs.They define the core functionality of the system, including inputs, processes, and outputs.

7.1 DOCUMENT UPLOAD

1. Upload PDF documents
2. Validate file format
3. Extract text

7.2 DOCUMENT PROCESSING

1. Chunk text
2. Maintain context

7.3 EMBEDDING GENERATION

1. Convert text into vectors
2. Store in DB

7.4 QUERY PROCESSING

1. Accept user query
2. Retrieve relevant chunks
3. Generate answer

8. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe how well a system should perform rather than what it does. They define the quality, performance, and constraints of the system.

8.1 SECURITY

1. Secure API keys
2. Safe file handling

8.2 PERFORMANCE

1. Fast retrieval
2. Efficient chunking

8.3 SCALABILITY

1. Supports large documents
2. Can scale to cloud

8.4 RELIABILITY

1. Consistent responses
2. Error handling in APIs

9. REFERENCES

1. OpenAI, "OpenAI API Documentation," 2025.
2. ChromaDB, "Chroma Documentation," 2025.
3. FastAPI, "FastAPI Documentation," 2025.
4. LangChain, "LangChain Documentation," 2025.
5. PDFPlumber, "PDF Processing Library," 2025.

GITHUB:https://github.com/Nainamanghani/GenAI_SolarEnergy/blob/main/requirements.txt